📝 Django Normativ 2-oy

7-Normativ

```
"""
📝 7-Normativ — Django Groups va Permissions
🕑 Bajarish uchun ajratilgan vaqt: 90 min
🔧 Mavzu: Django Groups, Permissions, Access Control

🎯 Maqsad:
• Django’da Group va Permission tizimi bilan ishlashni o‘rganish
• Foydalanuvchilarga rollar berish
• CRUD operatsiyalarini permission orqali nazorat qilish
• Faqat ruxsat berilgan foydalanuvchilar ma’lumotni o‘zgartira olishini ta’minlashh

------------------------------------------------------------

📌 Vazifa:

Mavjud loyiha (Post CRUD loyihasi) ichida foydalanuvchilar uchun
2 xil rol tashkil qiling:

1) Admin
2) User

Admin foydalanuvchilar barcha CRUD operatsiyalarini bajarishi mumkin.
User foydalanuvchilar esa faqat ma’lumotlarni ko‘ra oladi (Read).

------------------------------------------------------------
```

```
📌 1-bosqich: Group yaratish

Django admin panel orqali quyidagi group larni yarating:

1) Admin
2) User

Buning uchun:

1. Admin panelga kiring
2. Groups bo‘limiga o‘ting
3. "Add Group" tugmasini bosing
4. Avval "Admin" group yarating
5. Keyin "User" group yarating

------------------------------------------------------------

📌 2-bosqich: Permission berish

Post modeli uchun Django avtomatik ravishda quyidagi permissi
onlarni yaratadi:

• add_post
• change_post
• delete_post
• view_post

Endi group lar uchun permissionlarni belgilang.

Admin group uchun:
✔ add_post
✔ change_post
✔ delete_post
✔ view_post
```

```
User group uchun:
✔ view_post

Natijada:

Admin → to‘liq CRUD
User → faqat READ

------------------------------------------------------------

📌 3-bosqich: Userni groupga qo‘shish

Register qilingan foydalanuvchilarni avtomatik ravishda
User group ga qo‘shish kerak

User register bo‘lganda:

User group ga qo‘shiladi.

------------------------------------------------------------

📌 4-bosqich: Permission tekshirish

Create / Update / Delete view larni permission orqali
himoyalang.

Buni quyidagi usullar bilan qilish mumkin:

Misol:

@permission_required('posts.add_post')

------------------------------------------------------------

📌 5-bosqich: Template darajasida permission tekshirish
```

```
Template ichida ham permission tekshirish mumkin.

Misol:

Agar userda post qo‘shish huquqi bo‘lsa:

Create tugmasi chiqadi.

Agar bo‘lmasa:

tugma ko‘rinmaydi.

Misol:

{% if perms.posts.add_post %}
    Create button
{% endif %}

Shuningdek:

Update va Delete tugmalari ham permission asosida
ko‘rinishi kerak.

------------------------------------------------------------

📌 Kutiladigan natija:

Login qilgan user:

Admin bo‘lsa:
• Post qo‘shadi
• Post tahrir qiladi
• Post o‘chiradi
• Postlarni ko‘ra oladi

User bo‘lsa:
```

```
• Faqat postlarni ko‘ra oladi
• Create / Update / Delete tugmalari chiqmaydi

------------------------------------------------------------

📁 URL ko‘rinishi:

/posts/              → barcha userlar ko‘ra oladi
/posts/create/       → faqat Admin
/posts/<id>/edit/    → faqat Admin
/posts/<id>/delete/  → faqat Admin

------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"7-normativ: Group va Permission qo‘shildi"

------------------------------------------------------------

✅ Baholash mezonlari:

• 2 ta group mavjud (Admin, User)
• Permissionlar to‘g‘ri berilgan
• Register user avtomatik User group ga tushadi
• Permission view darajasida tekshirilgan
• Permission template darajasida ham tekshirilgan
• Admin CRUD qila oladi
• User faqat READ qila oladi
"""
```

8-Normativ

```
"""
📝 8-Normativ — Django .env va Signals
🕑 Bajarish uchun ajratilgan vaqt: 90 min
🔧 Mavzu: Environment Variables (.env), Django Signals

🎯 Maqsad:
• Django loyihada maxfiy ma’lumotlarni .env orqali boshqarish
• Environment variables ishlatish
• Django signal mexanizmini tushunish
• Model yaratish yoki user ro‘yxatdan o‘tishda avtomatik jara
yon ishlatish

---------------------------------------------------------------

📌 Vazifa:

Loyihaga quyidagi funksiyalarni qo‘shish:

1) .env fayl orqali maxfiy sozlamalarni boshqarish
2) Django Signal yozish
3) User register bo‘lganda signal ishlashi

---------------------------------------------------------------

📌 1-bosqich: django-environ o‘rnatish

Quyidagi paketni o‘rnating:

pip install

---------------------------------------------------------------

📌 2-bosqich: .env fayl yaratish
```

```
Loyiha root papkasida .env fayl yarating.

Misol:

SECRET_KEY=django-secret-key
DEBUG=True

POSTGRES_DB=
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_HOST=
POSTGRES_PORT=5432

---------------------------------------------------------------

📌 3-bosqich: settings.py sozlash

settings.py ichida:



---------------------------------------------------------------

📌 4-bosqich: .gitignore yangilash

.env fayl GitHub ga chiqmasligi kerak.

.gitignore ichiga qo‘shing:

.env

---------------------------------------------------------------

📌 5-bosqich: Django Signals yozish
```

```
Signals yordamida modelda sodir bo‘ladigan
hodisalarga avtomatik javob berish mumkin.

Masalan:

User yaratilganda signal ishlaydi.

---------------------------------------------------------------

📌 6-bosqich: signals.py fayl yaratish

accounts app ichida:

signals.py fayl yarating.

Signal quyidagicha ishlaydi:

User ro‘yxatdan o‘tganda:

• console ga log chiqaradi

---------------------------------------------------------------

📌 7-bosqich: post_save signal ishlatish

post_save signal ishlatiladi.

Bu signal model save bo‘lgandan keyin ishlaydi.

Misol:

User yaratildi → signal ishlaydi.

---------------------------------------------------------------

📌 8-bosqich: Signalni ulash
```

```
apps.py ichida signalni ulash kerak.

Misol:

ready() metod ichida:

signals import qilinadi.

Shundan keyin signal ishlay boshlaydi.

---------------------------------------------------------------

📌 Kutiladigan natija:

• Loyihada .env orqali maxfiy sozlamalar boshqariladi
• SECRET_KEY settings.py ichida yozilmaydi
• User register bo‘lganda signal ishlaydi
• Console log  amal bajariladi

---------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"8-normativ: .env va Signals qo‘shildi"

---------------------------------------------------------------

✅ Baholash mezonlari:

• .env fayl ishlatilgan
• settings.py environment variables orqali sozlangan
• .env GitHub ga push qilinmagan
```

```
• Django signal yozilgan
• Signal to‘g‘ri ishlayapti
"""
```

9-Normativ

```
"""
📝 9-Normativ — Django File Upload (Image Upload)
🕑 Bajarish uchun ajratilgan vaqt: 90 min
🔧 Mavzu: ImageField, MEDIA files, File upload

🎯 Maqsad:
• Django’da fayl yuklashni o‘rganish
• ImageField bilan ishlash
• MEDIA fayllarni sozlash
• Yuklangan rasmni template’da ko‘rsatish

---------------------------------------------------------------

📌 Vazifa:

Post loyihasiga rasm yuklash funksiyasini qo‘shing.

Har bir Post uchun:

• rasm yuklash mumkin bo‘lsin
• rasm list sahifada ko‘rinsin
• rasm detail sahifada ko‘rinsin

---------------------------------------------------------------

📌 1-bosqich: Pillow o‘rnatish
```

```
Django’da ImageField ishlashi uchun
Pillow kutubxonasi kerak.

pip install pillow

------------------------------------------------------------

📌 2-bosqich: Modelni yangilash

Post modelga yangi maydon qo‘shing:

image = models.ImageField(
    upload_to='posts/',
    null=True,
    blank=True
)

------------------------------------------------------------

📌 3-bosqich: Migratsiya qilish

python manage.py makemigrations
python manage.py migrate

------------------------------------------------------------

📌 4-bosqich: MEDIA sozlash

settings.py ichida:

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

------------------------------------------------------------

📌 5-bosqich: urls.py sozlash
```

```
config/urls.py ichida:

from django.conf import settings
from django.conf.urls.static import static



---------------------------------------------------------------

📌 6-bosqich: Formni yangilash

Agar ModelForm ishlatilayotgan bo‘lsa:

image field formda ko‘rinishi kerak.

---------------------------------------------------------------

📌 7-bosqich: Template yangilash

Create / Update sahifalarda:

<form method="post" enctype="multipart/form-data">

enctype yozilishi shart.

---------------------------------------------------------------

📌 8-bosqich: Rasmni chiqarish

List yoki detail sahifada:

Agar postda rasm bo‘lsa:

rasm ko‘rsatiladi.
```

```
Agar rasm bo‘lmasa:

hech narsa chiqmaydi.

------------------------------------------------------------

📌 Kutiladigan natija:

• Post yaratishda rasm yuklash mumkin
• Yuklangan rasm media papkaga saqlanadi
• Rasm list sahifada chiqadi
• Rasm detail sahifada chiqadi

------------------------------------------------------------

📁 URL ko‘rinishi:

/posts/
/posts/create/
/posts/<id>/

------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"9-normativ: Image upload qo‘shildi"

------------------------------------------------------------

✅ Baholash mezonlari:

• Pillow o‘rnatilgan
• ImageField modelga qo‘shilgan
```

```
• MEDIA sozlamalari ishlayapti
• Rasm yuklash ishlayapti
• Template’da rasm ko‘rinmoqda
"""
```

10-normativ

```
"""
📝 10-Normativ — Custom Forgot Password (Verification Code orqali)
🕑 Bajarish uchun ajratilgan vaqt: 90–120 min
🔧 Mavzu: Email yuborish, verification code, password reset

🎯 Maqsad:
• Django’da custom forgot password tizimi yozish
• Email orqali verification code yuborish
• Code orqali foydalanuvchini tekshirish
• Yangi parol o‘rnatish

❗ Muhim:
Django built-in password reset ishlatilmaydi.
Barcha logika o‘zimiz yozamiz.

------------------------------------------------------------

📌 Vazifa:

Loyihaga forgot password funksiyasini qo‘shing.

Jarayon quyidagicha ishlaydi:

1) Foydalanuvchi username kiritadi
2) Emailga verification code yuboriladi
```

```
3) Foydalanuvchi code va yangi parol kiritadi
4) Parol yangilanadi

------------------------------------------------------------

📌 1-bosqich: Code modeli

Verification code saqlash uchun model yarating.

Model maydonlari:

• code (PositiveIntegerField)
• user (ForeignKey User)
• expired_date (DateTimeField)

Code avtomatik generatsiya qilinishi kerak.

generate_code() funksiyasi orqali:

100000 – 999999 oralig‘ida random code yaratiladi.

------------------------------------------------------------

📌 2-bosqich: Code expiration

Code cheksiz ishlamasligi kerak.

exp_time_now() funksiyasi orqali:

Code faqat 2 daqiqa ishlaydi.

expired_date = timezone.now() + timedelta(minutes=2)

------------------------------------------------------------

📌 3-bosqich: ForgotPassword View
```

```
Foydalanuvchi username kiritadi.

View jarayoni:

1) user topiladi
2) yangi Code yaratiladi
3) emailga code yuboriladi

Email ichida:

verification code bo‘ladi.

------------------------------------------------------------

📌 4-bosqich: Email yuborish

Email yuborish uchun:

send_email_thread funksiyasidan foydalaniladi.

Email tarkibi:

Subject: Parolni tiklash

Message:
Code: 123456

------------------------------------------------------------

📌 5-bosqich: RestorePassword View

User quyidagilarni kiritadi:

• verification code
• new password
```

```
• confirm password

Form ichida tekshiriladi:

1) code to‘g‘rimi
2) code eskirib ketmaganmi
3) passwordlar bir xilmi

------------------------------------------------------------

📌 6-bosqich: Parolni yangilash

Agar barcha tekshiruvlar to‘g‘ri bo‘lsa:

user.set_password(new_password)

user.save()

Foydalanuvchi login sahifaga redirect qilinadi.

------------------------------------------------------------

📌 Kutiladigan natija:

• Forgot password sahifasi mavjud
• Email orqali verification code keladi
• Code orqali tekshirish ishlaydi
• Yangi parol o‘rnatiladi
• Code 2 daqiqadan keyin ishlamaydi

------------------------------------------------------------

📁 URL ko‘rinishi:

/forgot-password/
/restore-password/
```

```
---------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"10-normativ: Custom forgot password qo‘shildi"

---------------------------------------------------------------

✅ Baholash mezonlari:

• Code modeli mavjud
• Code random generatsiya qilinadi
• Email yuborish ishlayapti
• Code expiration ishlayapti
• Password reset ishlayapti
"""
```

11-normativ

```
"""
📝 11-Normativ — Django Custom Middleware (Request Logger)
🕒 Bajarish uchun ajratilgan vaqt: 90 min
🔧 Mavzu: Django Middleware, Request logging

🎯 Maqsad:
• Django’da custom middleware yozishni o‘rganish
• Har bir requestni nazorat qilish
• User faoliyatini log faylga yozish
```

```
------------------------------------------------------------

📌 Vazifa:

Loyihaga custom middleware qo‘shing.

Middleware quyidagilarni log faylga yozishi kerak:

• request qilingan URL
• request vaqti
• foydalanuvchi username
• foydalanuvchi IP manzili

------------------------------------------------------------

📌 1-bosqich: Middleware fayl yaratish

accounts ichida:

middleware.py fayl yarating.

Bu faylda custom middleware klass yoziladi.

------------------------------------------------------------

📌 2-bosqich: Middleware vazifasi

Har bir request kelganda:

Quyidagi ma’lumotlar olinadi:

• request.path
• request.user
• request.META['REMOTE_ADDR']
• timezone.now()
```

```
Bu ma’lumotlar log faylga yoziladi.

------------------------------------------------------------

📌 3-bosqich: Log fayl yaratish

Loyiha root papkasida:

requests.log

fayl yarating.

Har bir request shu faylga yozilishi kerak.

------------------------------------------------------------

📌 4-bosqich: Middleware ni ulash

settings.py ichidagi

MIDDLEWARE ro‘yxatiga
custom middleware qo‘shiladi.

Masalan:

"accounts.middleware.RequestLoggerMiddleware"

------------------------------------------------------------

📌 5-bosqich: Middleware ishlash tartibi

Middleware quyidagicha ishlaydi:

1) request keladi
2) middleware ishlaydi
3) log faylga yozadi
```

```
4) keyin view ishlaydi

---------------------------------------------------------------

📌 Log yozuvi misoli:

[2026-03-10 14:25]
User: giyos
IP: 127.0.0.1
Path: /posts/create/

---------------------------------------------------------------

📌 Kutiladigan natija:

• Har bir request log faylga yoziladi
• Login qilgan user username chiqadi
• Login qilmagan user "AnonymousUser" bo‘ladi
• URL va vaqt yoziladi

---------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"11-normativ: Custom middleware qo‘shildi"

---------------------------------------------------------------

✅ Baholash mezonlari:

• Custom middleware yozilgan
• settings.py ga qo‘shilgan
• Log fayl yaratilgan
```

```
• Har bir request log qilinmoqda
• Username, IP, path yozilmoqda
"""
```

12-normativ final

```
"""
📝 12-Normativ — Localization va OAuth2 (Google Login)
🕑 Bajarish uchun ajratilgan vaqt: 120 min
🔧 Mavzu: Django Localization, OAuth2, Social Login

🎯 Maqsad:
• Django'da ko'p til qo'llab-quvvatlash (Localization)
• gettext yordamida matnlarni tarjima qilish
• Google orqali login qilish (OAuth2)
• Social authentication bilan ishlash

------------------------------------------------------------

📌 Vazifa:

Loyihaga quyidagi funksiyalarni qo'shing:

1) Localization (2 ta til)
2) Google orqali login qilish (OAuth2)

------------------------------------------------------------

📌 1-bosqich: Localization sozlash

settings.py ichida:

LANGUAGE_CODE = 'en'
```

```
LANGUAGES = [
    ('en', 'English'),
    ('uz', 'Uzbek'),
]

USE_I18N = True

------------------------------------------------------------

📌 2-bosqich: Locale middleware qo‘shish

settings.py ichida MIDDLEWARE ga qo‘shing:

'django.middleware.locale.LocaleMiddleware'

------------------------------------------------------------

📌 3-bosqich: Matnlarni tarjima qilish

Template ichida:

{% trans "Post List" %}

yoki

{% trans "Login" %}

Python kodida:

from django.utils.translation import gettext as _

_("Login")

------------------------------------------------------------
```

```
📌 4-bosqich: Tarjima fayllarini yaratish

Quyidagi komanda ishlatiladi:

django-admin makemessages -l uz

------------------------------------------------------------

📌 5-bosqich: Tarjima yozish

locale/uz/LC_MESSAGES/django.po fayl ichida
matnlar tarjima qilinadi.

------------------------------------------------------------

📌 6-bosqich: Tarjimani aktiv qilish

Quyidagi komanda ishlatiladi:

django-admin compilemessages

------------------------------------------------------------

📌 7-bosqich: Til almashtirish

Template ichida til almashtirish imkoniyati
bo‘lishi kerak.

Masalan:

English / Uzbek tugmalari.

------------------------------------------------------------

📌 8-bosqich: OAuth2 (Google Login)
```

```
Google orqali login qilish funksiyasini qo‘shing.

Buning uchun quyidagi kutubxonadan foydalanish mumkin:

django-allauth

------------------------------------------------------------

📌 9-bosqich: Google OAuth sozlash

Google Cloud Console orqali:

1) OAuth Client yaratish
2) Client ID olish
3) Client Secret olish

------------------------------------------------------------

📌 10-bosqich: Django sozlash

settings.py ichida Google OAuth sozlamalari
qo‘shiladi.

------------------------------------------------------------

📌 11-bosqich: Login sahifaga Google login qo‘shish

Login sahifada:

"Login with Google"

tugmasi bo‘lishi kerak.

------------------------------------------------------------

📌 Kutiladigan natija:
```

```
• Sayt 2 ta tilni qo‘llab-quvvatlaydi
• Til almashtirish ishlaydi
• Matnlar tarjima qilinadi
• Foydalanuvchi Google orqali login qila oladi

------------------------------------------------------------

📁 Topshirish:

• GitHub repository link
• Commit message:

"12-normativ: Localization va Google OAuth qo‘shildi"

------------------------------------------------------------

✅ Baholash mezonlari:

• Localization ishlayapti
• Til almashtirish ishlayapti
• Tarjima fayllari yaratilgan
• Google login ishlayapti
• OAuth to‘g‘ri sozlangan
"""
```